Final Project

David Ho [hod12], Alex Horton [hortoa5]

April 4th, 2026

Contents

0.1 Executive Summery 3
0.2 Part 1 4
0.2.1 Approximations 4
0.2.2 Mystery 7
0.3 Part 2 8
0.4 Part 3 9
0.5 Part 4 12
0.5.1 More robust 12
0.5.2 Other types of Graph 12
0.6 Appendix 13

List of Figures

1 4
2 4
3 5
4 6
5 6
6 7
7 8
8 9
9 9

0.1 Executive Summery

- We built further understanding of Dijsktra's and Bellman-Ford's shortest path algorithms and how limiting the number of relaxations affects these algorithms.
- Gained further understanding of all-pairs shortest path algorithms and their runtime.
- Learned what the A* algorithm is and how it worked
- Learned when A* is better than Dijkstra's and when to use it
- We built upon this knowledge by applying it to a real world example of the London Train System
- Gained further understanding of object oriented programming techniques in Python
- Learned how object oriented programming techniques can help organize code into meaningful layers

0.2 Part 1

0.2.1 Approximations

Experiment 1

For this experiment, the number of nodes was varied to check the impact it has on the approximation, the metric being the total cost. Experimental outline: 5 random complete graphs for each number of nodes ranging from 1 to 150 (skipping every 5), and edge weights from 1 to $2 \times numberofnodes$ (to try and avoid duplicate weights) were generated. The approximations and actual total distance (using Dijkstra) were computed and averaged for each of these number of nodes. The experiment was run 5 times before the following results were collected

Figure 1:

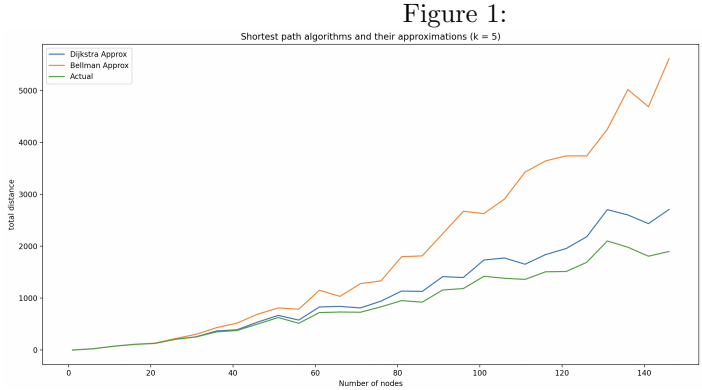
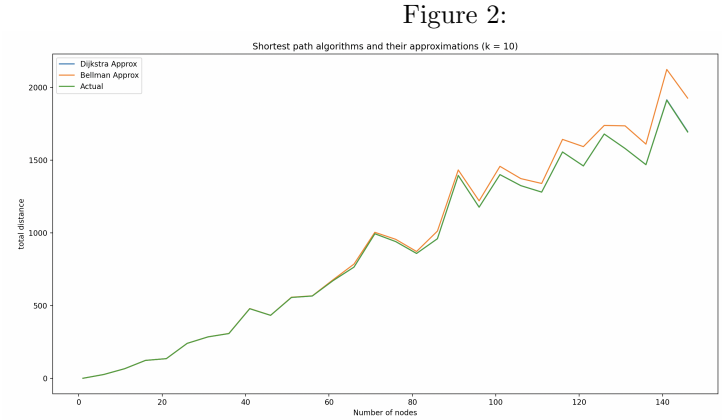


Figure 2:



Based on figure 1, which had a fixed k value of 5 there seems to be some sort of relation between number of nodes and approximation. Since as the number of

nodes increased both of the approximations got worse, with bellman-ford being significantly worse than the actual. However, I also ran the experiment with a k value of 10, and the results contradict this initial assumption. Based on Figure 2, it seems to be that the number of relaxations has an impact on the performance of the approximations, not necessarily the number of nodes. This possibility will be later explored in experiment 3.

Experiment 2

For this experiment, the number of edges was varied to check the impact it has on the approximation, the metric being the total cost. Experimental outline: 5 random graphs with 50 nodes were generated for each number of edges ranging from 1 to 50(50 - 1) = 2450 (max number of edges) (skipping every 10), and edge weights from 1 to 2×50 (to try and avoid duplicate weights). The approximations and actual total distance (using Dijkstra) were computed and averaged for each of these number of edges. The experiment was run 5 times before the following results were collected

Figure 3:

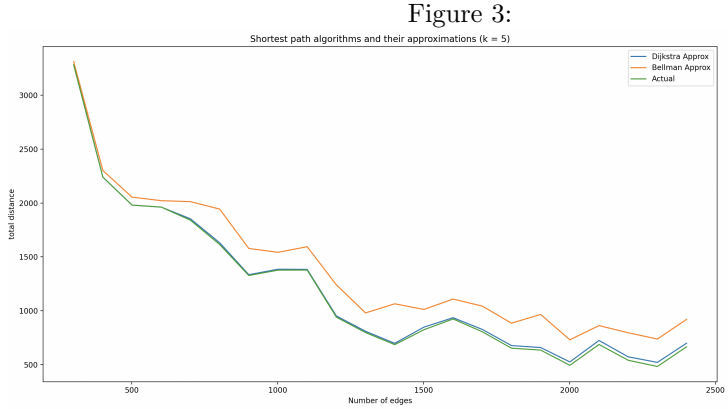
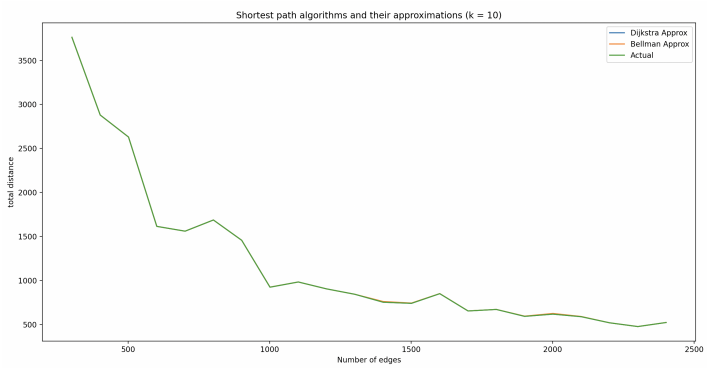


Figure 4:

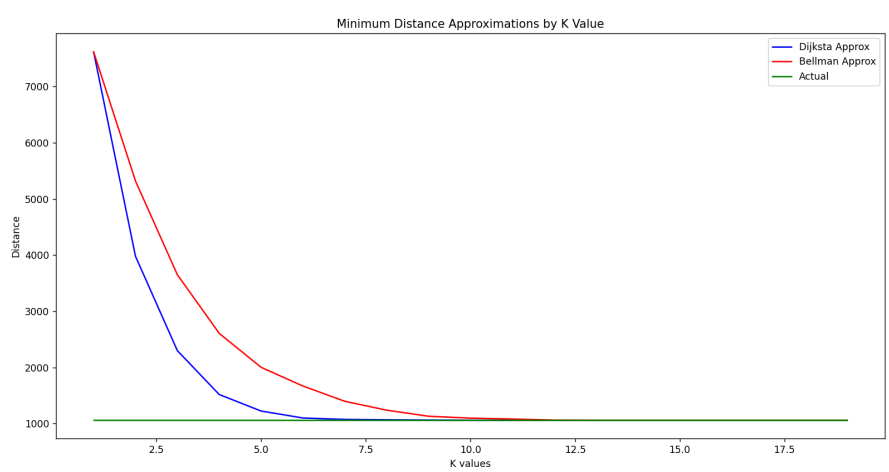


Similar to experiment 1, the number of edges does not really seem to impact the approximations efficacy all that much, and it seems to be only the k value that impacts this. This is demonstrated by the number of edges varying, but the approximation being almost identical to the actual for a k value of 10, meanwhile there is more variance between approximation and actual for a k value of 5.

Experiment 3

For this experiment I created 10 random graphs with 75 nodes and max edge weight of 200. I ran the Bellman-Ford and Dijkstra's approximation algorithm on all ten graphs and took the average for each k value. I ran this for k values 1 to 19. I ran this 5 times before I recorded the current data.

Figure 5:

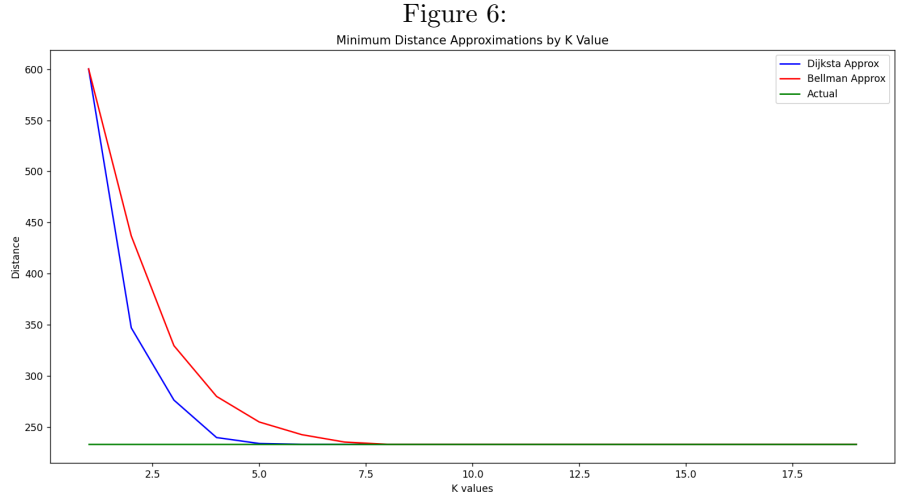


As you can see, the Dijkstra's Approximation matches the actual around k

value 6 and the Bellman-Ford Approximation matches the actual later at around k value 12.

For this experiment I created 10 random graphs with 20 nodes and max edge weight of 60. I ran the Bellman-Ford and Dijkstra's approximation algorithm on all ten graphs and took the average for each k value. I ran this for k values 1 to 19. I ran this 5 times before I recorded the current data.

Figure 6:



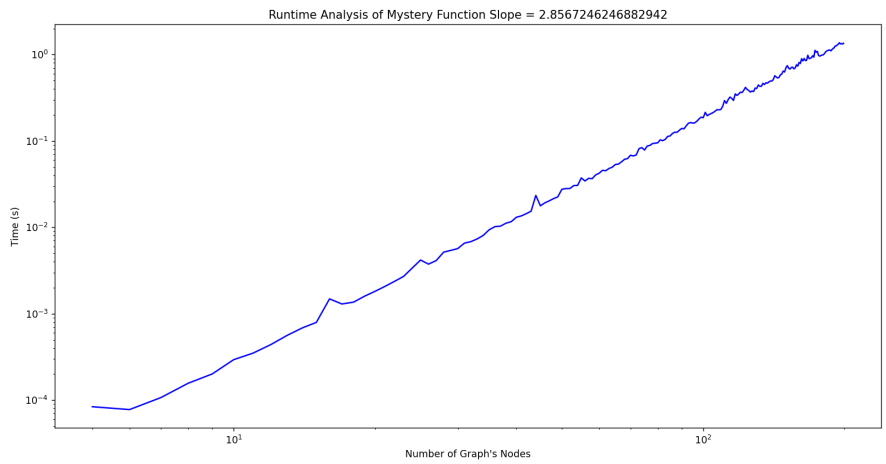
The goal of running this experiment again with graphs of 20 nodes and max weight 60, was to see if the number of nodes affects the k value needed to match the actual distance. In the figure above, we can see that this does affect our k values. Now Dijkstra's matches at about k value 5 and Bellman-Ford matches at about k value 8. This confirms experiments 1's conjecture.

0.2.2 Mystery

First, we consider the two different implementations of the "all-pairs shortest path algorithm". One uses Dijkstra's algorithm and the other uses Bellman-Fords algorithm. Since for both algorithms we simply run Dijkstra's or Bellman-Ford algorithm for every node we multiply the current time complexity by V for both. So the one that implements Dijkstra's is $\Theta(v^3)$ and the Bellman-Ford implementation is $\Theta(v^4)$ for dense graphs.

For this experiment, I created 195 random graphs with increasing nodes from 5 to 200. I ran the mystery function for each graph and measured the time it took to complete. I then graphed the time to complete and number of nodes on a log-log graph. I ran this 5 times before the current data was taken.

Figure 7:



The slope of this graph is ~ 2.86. I noticed this gets closer and closer to 3 as the number of nodes increases. In a log-log graph the slope signifies the exponent in the polynomial form $y = ax^n$. This means that the time complexity of this algorithm is likely $\Theta(v^3)$. This matches our theoretical time complexity for the Dijkstra's implementation of our "all-pairs shortest path algorithm". I also noticed that the mystery function works with negative weight edges but not with negative cycles which is what you would expect from the Dijkstra's implementation. So in conclusion, the mystery function is likely the Dijkstra's implementation of our "all-pairs shortest path algorithm".

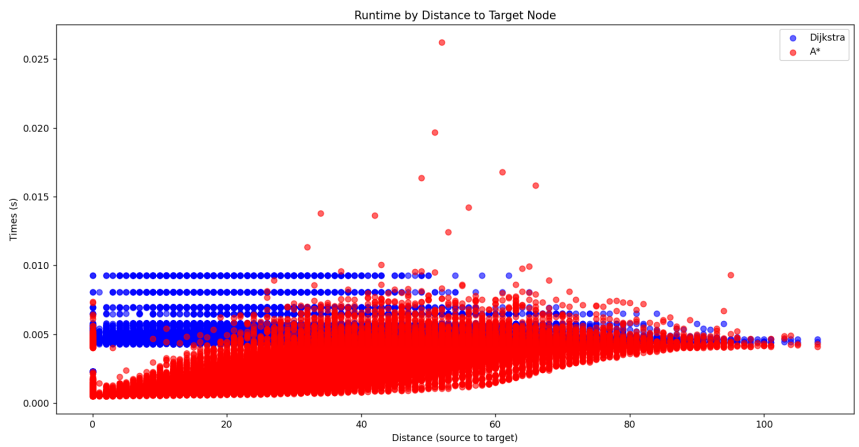
0.3 Part 2

- The issue with Dijkstra's algorithm is that it explores all directions equally. This means that it wastes time exploring paths leading away from the goal. A* tries to address this by having a heuristic function. This function tells it the general direction it should be going.
- To test the difference between A* and Dijkstra's algorithm we would likely need a real world example so that we could create a meaningful heuristic function. We could measure the number of nodes visited(number of extract min operations) and time taken to reach the same goal on different graphs.
- If your random heuristic function overestimates the true distance than your A* algorithm may not find the true shortest path at all. With a random heuristic it may not help at all and actually worst case it does worse than Dijkstra's because it still needs to do the extra adding operation to each priority calculation.
- GPS may use A* by creating a straight line to the location and basing the heuristic off of that. Also NPC path finding can use A* using the euclidean distance as your heuristic.

0.4 Part 3

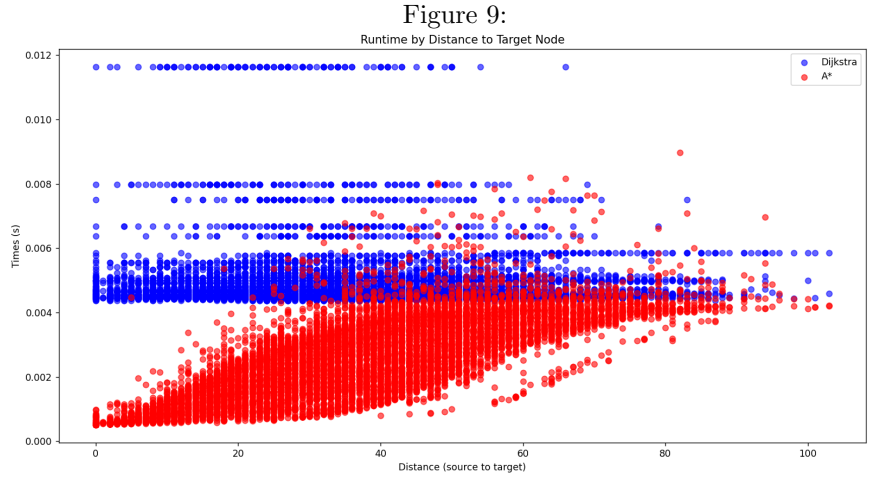
For this experiment I ran Dijkstra's and A* algorithm on the London railway station map. The weight of each edge is determined by the time variable in the connections file. I created the heuristic function using the euclidean distance from one station to another based on their latitude and longitude multiplied by a factor of 25 so that the results were significant enough to change the path A* takes. I ran this for all possible pairs and plotted it using a scatter plot. I ran this 5 times before taking the current data.

Figure 8:



In the figure below I reduced the number of stations we consider to reduce the number of outliers and get cleaner data.

Figure 9:



You will notice that Dijkstra's points follow horizontal lines. This is because the target station does not affect the runtime of Dijkstra's algorithm but it does affect the A* algorithm. In the graphs you can see as the distance between the stations gets higher, the closer Dijkstra's and A* get to each other in runtime. This is rather un-intuitive because one would expect A* to perform better the further apart the stations are. However, in this case, what is likely happening is that due to the confined space of the city, stations that are the furthest apart are often near the edges of the map. Thus, the benefit of A* directed search over Dijkstra's expanding search is minimized as there is no room for Dijkstra's to go the wrong direction or overshoot the target.

Notes about certain combinations

For each of the scenarios (Same line, adj line, transfers) 3 source and targets pairs were chosen and the runtime and length of this path were recorded and averaged to compute a total length and total time for each of the cases. Both dijkstra and A* were run and the results were compared. Experiment was run 5 times before results were taken. The following were the stations chosen for each case:

Same line

Southgate (232) → Manor house (160)
South ruislip (239) → North acton (181)
Wimbledon (299) → West brompton (287)

Adj line

Adj pair 1:
Hammersmith(110) → Westbourne Park(283)
Hammersmith(110) → Ladbroke grove(147)

Adj pair 2:
Liverpool St(156) → Euston Square(90)
Liverpool St(156) → Great Portland St(104)

Adj pair 3:
Ealing broadway(72) → West Kensington(293)
Ealing broadway(72) → Barons court(17)

Transfer (At least 2 transfers)

Queen's Park (206) → South kensington (236)
Bank (13) → Baker Street(110)
Wimbledon(299) → Bank(13)

Results

Same line data:
Dijkstra Same line (weight): 13.6667
Dijksttra Same line (runtime): 2.2367ms
A_star Same line (weight): 13.6667
A_star Same line (runtime): 0.2968ms

Adjacent line data:
Dijkstra Adj line (weight): 10.6667
Dijksttra Adj line (runtime): 2.1918ms
A_star Adj line (weight): 10.6667
A_star Adj line (runtime): 0.4745ms

Transfer data:
Dijkstra Transfer (weight): 25.3333
Dijksttra Transfer (runtime): 2.0106ms
A_star Transfer (weight): 25.3333
A_star Transfer (runtime): 0.773ms

Explanation of Results

Same line
For the same line, Dijkstra and A* computed the same average shortest path for the chosen stations. However, A* significantly outperformed Dijkstra in runtime, since Dijkstra had to compute the shortest path to every other node.
Adjacent lines
Same as above, Dijkstra and A* computed the same average shortest path for the chosen stations. However, again, A* significantly outperformed Dijkstra in runtime. Although A* was a little slower than the same line experiment, but Dijkstra was a little bit faster (than their respective counterparts)
Transfer lines
Again, A* and Dijkstra computed the same average shortest path for the chosen stations. And again A* also outperformed Dijkstra in terms of runtime, but it is now slower than the same line and adjacent line version. However, Dijkstra transfer was faster than the other previous versions.

Summary Since Dijkstra computes the shortest path for every node of the graph, the runtime isn't really needed for all 3 cases as it can vary each time it is run, so it isn't really fair to compare them for the different cases. However, in all cases A* gets the shortest path, and does it significantly faster. In the following order (from fastest to slowest): same line, adjacent line, transfer lines. Transfer is the worst of the 3, most likely due to the algorithm needing to check more paths instead of a direct path (same and adjacent).

0.5 Part 4

Private fields were implemented using the __ prefix, ex self.__heuristic = h

A* Class

I treated SPAlgorithm as an Abstract class with abstract method calc_sp that the 3 shortest path algorithms have to provide an implementation for.
However, calc_sp only takes arguments (graph, source, dest) but A* requires a heuristic argument.
Assuming you call the method with a HeuristicGraph, then within this function you can retrieve said heuristic by using that classes getter method.

Design principles

Encapsulation - Graph and its subclasses encapsulate all required fields and methods. Also the SPAlgorithm and its subclases encapsulate all their required methods (calc_sp).
Abstraction - SPAlgorithm Abstract class hides implementation details within its subclasses.
Inheritance - WeightedGraph inherits from Graph, and HeuristicGraph inherits from WeightedGraph. Also the shortest path algorithms inherit from SPAlgorithm.
Polymorphism - Shortest path algorithms override calc_sp
Finally, the Class ShortPathFinder is composed of a Graph and SPAlgorithm, and it can't exist without either one.
Patterns: Adapter

0.5.1 More robust

You should decouple the node type, where instead of hardcoding the methods for a specific node type. Allow it so nodes can be of different types, and handle this when implementing via overloading the function (if different behaviour is required). Or you could define a Node class and have it so Graphs are composed of Nodes. This Node class can have a node id and value if required, which can then be stored inside the Graph implementation.
Essentially I would add overloaded methods to handle for different types of nodes, or create a Node class and make it so graphs are composed of them instead of a single data type.

0.5.2 Other types of Graph

My implementation used an Adjacency list approach, however you could also have done an Adjacency matrix approach, or Edge list approach.

0.6 Appendix

You can find parts 1-2 and the experiment and file processing for part 3 in $final_project_part1.py$ file, all of which is below the comment that says "our code".

Some of part 3 (graphs used) and all of part 4 is in the $part3_4.py$ file.